

|                                                                                   |                            |              |
|-----------------------------------------------------------------------------------|----------------------------|--------------|
|  | <b>Atividade — Aula 36</b> |              |
|                                                                                   | <b>NOME:</b>               |              |
|                                                                                   | <b>TURMA:</b>              |              |
|                                                                                   | <b>TURNO:</b>              | <b>NOTA:</b> |
| <b>DATA:</b>                                                                      |                            |              |

## Parte 1 — Conceitos: Vulnerabilidades

1. O que é o OWASP Top 10? Cite 3 vulnerabilidades.

É uma lista das 10 vulnerabilidades mais críticas em aplicações web, mantida pela OWASP.

- Injection – atacante insere código malicioso (SQL, etc.) em campos de entrada
- Broken Authentication – sessões ou senhas mal protegidas permitem acesso indevido
- XSS – atacante injeta scripts no navegador de outros usuários
- 

2. O que é XSS? Como é explorado? Diferença entre `<%- %>` e `<%= %>` no EJS?

XSS é quando o atacante insere um `<script>` numa entrada e o navegador de outro usuário executa esse código. Ex: roubar cookies, redirecionar páginas.

- `<%- %>` → renderiza HTML bruto (perigoso, permite XSS)
- `<%= %>` → escapa o HTML (seguro, vira texto puro)

3. Por que validação no front-end não basta?

Porque o atacante pode usar Thunder Client, curl ou qualquer ferramenta para mandar requisições direto pro servidor, ignorando completamente o HTML. O `required` e `minlength` só existem no navegador.

4. O que é IDOR? Exemplo prático.

IDOR é quando você acessa um recurso de outra pessoa só trocando o ID na URL.

Exemplo: usuário A está logado e acessa `/comentarios/42/remover` sendo que o comentário 42 é do usuário B. Se o servidor não checar o dono, remove mesmo assim.

5. Por que a mensagem de login deve ser genérica?

Se você diz "Email não encontrado", o atacante sabe que aquele email não existe e pode tentar outros. Com "Email ou senha incorretos", ele não descobre nada.

6.

5 dados que nunca devem ir na resposta da API.

- Senha (mesmo em hash)
- Token de sessão

- CPF / dados pessoais sensíveis
  - Perguntas/respostas de segurança
  - Role/permisões internas (quando não necessário)
- 

7. Diferença entre `httpOnly: true` e `false`. Qual ataque previne?

- `httpOnly: false` → JavaScript da página consegue ler o cookie (`document.cookie`)
- `httpOnly: true` → JavaScript não consegue ler o cookie, só o navegador envia automaticamente

Previne o ataque de roubo de sessão via XSS.

---

8. O que é sanitização? 3 exemplos.

É limpar/transformar os dados antes de salvar, independente do que o usuário mandou.

- `trim()` → remove espaços do início e fim
  - `escape()` → converte `<script>` em `&lt;script&gt;`
  - `normalizeEmail()` → converte `ANA@Gmail.Com` em `ana@gmail.com`
- 

9. O que é `express-validator`? Vantagem sobre `if` manual.

É uma biblioteca que valida e sanitiza campos do `req.body` de forma declarativa.

Vantagem: `body("campo").trim().notEmpty()` é encadeável, reutilizável e já retorna mensagens de erro padronizadas. O `if` manual é repetitivo e fácil de esquecer algum campo.

---

10. Diferença entre validação e sanitização no `express-validator`.

- Validação (verifica formato, não altera):
    - `.isEmail()` → checa se é email válido
    - `.isLength({ min: 6 })` → checa tamanho mínimo
  - Sanitização (limpa/transforma):
    - `.trim()` → remove espaços
    - `.normalizeEmail()` → padroniza o email
- 

11. O que o `Helmet.js` faz? 3 headers que ele configura.

Adiciona headers HTTP de segurança automaticamente com uma linha: `app.use(helmet())`.

- `X-Content-Type-Options: nosniff` → impede o browser de "adivinhar" o tipo do arquivo
  - `X-Frame-Options: DENY` → impede que a página seja colocada em `iframe` (clickjacking)
  - `Strict-Transport-Security` → força HTTPS
- 

12. O que é `rate limiting`? Por que login é mais restrito?

Limita quantas requisições um IP pode fazer num período. Login é mais restrito (5/15min) porque é o alvo de ataques de força bruta (testar milhares de senhas). Rotas gerais só precisam de proteção contra abuso, não precisam ser tão rígidas.

---

### 13. O que é CORS? Quando precisa configurar?

CORS é uma política do navegador que bloqueia requisições de origens diferentes. Você precisa configurar quando o front e o back estão em domínios/portas diferentes (ex: front na 5173, back na 3000). Sem configurar, o browser bloqueia a resposta e a requisição falha.

---

### 14. O que é SQL Injection? Por que queries parametrizadas previnem?

Atacante insere SQL num campo: ' OR '1'='1 e manipula a query. Queries parametrizadas (\$1, \$2) separam o código SQL dos dados — o banco trata os parâmetros como texto puro, nunca como código. Mesmo usando JSON vale saber pois pode usar um banco SQL no futuro.

---

### 15. Ordem correta dos middlewares no app.ts. Por que segurança vem antes?

```
helmet()    // segurança
cors()      // controle de origem
rateLimiter // limita requisições
express.json() // parsers
session()   // sessão
rotas      // lógica
```

Segurança vem antes porque se o parser vier primeiro, dados maliciosos já foram processados antes de qualquer proteção agir.

---

### 16. Corrigir XSS no EJS.

```
ejs
<!-- errado -->
<h1>Bem-vindo, <%- usuario.nome %>!/</h1>
<p>Seu comentário: <%- comentario.texto %></p>
```

```
<!-- correto -->
<h1>Bem-vindo, <%= usuario.nome %>!/</h1>
<p>Seu comentário: <%= comentario.texto %></p>
```

Troca <%- por <%= para escapar o HTML.

---

### 17. 2 problemas de segurança no login.

```
js
const user = users.find(u => u.email === email && u.senha === senha);
if (!user) { res.send("Email não encontrado"); return; }
  1. Compara senha em texto puro — deveria usar bcrypt.compare()
  2. Mensagem específica "Email não encontrado" — deveria ser genérica
```

---

### 18. 3 vulnerabilidades no registro.

js

```
users.push({ id: users.length+1, nome, email, senha, role: "user" });
```

1. Sem validação — campos podem vir vazios ou com formato inválido
  2. Senha em texto puro — deveria usar `bcrypt.hash()`
  3. Sem checar email duplicado — pode cadastrar o mesmo email várias vezes
- 

#### 19. IDOR no comentário. Problema e correção.

```
js
// problema: qualquer um pode remover qualquer comentário
app.post("/comentarios/:id/remover", (req, res) => {
  comentarios = comentarios.filter(c => c.id !== Number(req.params.id));
  Correção: verificar se o comentário pertence ao usuário logado antes de remover.
js
comentarios = comentarios.filter(
  c => !(c.id === Number(req.params.id) && c.userId === req.session.userId)
);
```

---

#### 20. API expondo dados sensíveis. Problema e correção.

```
js
// problema: retorna todos os campos incluindo senha
res.json(users);

// correção: remove a senha antes de enviar
res.json(users.map(({ senha, ...rest }) => rest));
```

---

#### 21. Vulnerabilidade na configuração da session.

```
js

secret: "123"      // fraco, fácil de adivinhar
httpOnly: false    // JavaScript pode ler o cookie → vulnerável a XSS
secure: false      // cookie trafega em HTTP → vulnerável a interceptação
```

---

#### 22. Valor de `req.body.email` após a validação.

O usuário digitou " ANA@Gmail.Com ". Após `isEmail().normalizeEmail()` o valor vira: `ana@gmail.com`  
Remove espaços, converte para minúscula e padroniza o domínio.

---

#### 23. Problema no rate limiter.

```
js
windowMs: 1000, // 1 segundo
max: 1,         // 1 requisição
app.use(limiter); // aplicado em TODAS as rotas
```

Problema: 1 req/segundo para tudo. Carregar uma página com CSS, JS e imagens dispara várias requisições ao mesmo tempo → usuário legítimo é bloqueado na hora.

---

24. O cors() sem configuração está correto para produção?

Não. app.use(cors()) sem config aceita requisições de qualquer origem (\*). Em produção deve restringir para o domínio do front:

```
js
app.use(cors({ origin: "https://meusite.com" }));
```

---

25. Problema na validação da nota.

```
js
body("nota").isFloat({ min: 0, max: 10 })
```

Se o campo nota não existir no body, o isFloat recebe undefined e pode passar ou dar erro inesperado. Falta adicionar .notEmpty() ou .exists() antes:

```
js
body("nota").exists().isFloat({ min: 0, max: 10 })
```

---

26. Problema na ordem dos middlewares.

```
js
app.use(authRoutes);    // rotas primeiro
app.use(express.json()); // parser depois
app.use(helmet());      // segurança por último
```

Problema: as rotas são registradas antes do parser, então req.body está vazio quando chega nas rotas. E o helmet não protege nada pois vem depois. A ordem certa é: helmet → parsers → rotas.

---

### Parte 3A — Prática: Caçar e corrigir vulnerabilidades

Use o projeto app-seguro fornecido. Ele tem 11 VULNERABILIDADES PROPOSITAIS.

27. Execute o projeto e TESTE cada ataque:

- ☐ XSS: digitar <script>alert("XSS")</script> como comentário → alert apareceu?
- ☐ IDOR: criar comentário como usuário A, remover como usuário B
- ☐ API: acessar /api/usuarios no navegador → senhas visíveis?
- ☐ Admin: logar como user normal e acessar /admin
- ☐ Registro: enviar POST com campos vazios via Thunder Client

28. Corrija as vulnerabilidades de session:

- a) Trocar secret "123" por string longa
- b) Trocar httpOnly: false por httpOnly: true

29. Corrija as vulnerabilidades de login/registro:

- a) Instalar bcrypt → bcrypt.compare no login

- b) bcrypt.hash no registro (nunca texto puro)
- c) Mensagem genérica: "Email ou senha incorretos"
- d) Validação: nome, email (@), senha (min 6)
- e) Verificar email duplicado antes de registrar

**30. Corrija as vulnerabilidades de dados:**

- a) IDOR: verificar userId em editar e remover
- b) API: filtrar senha → users.map(({ senha, ...rest }) => rest)
- c) Admin: adicionar verificação de role
- d) XSS: trocar <%- c.texto %> por <%= c.texto %> na view

### Parte 3B — Prática: Adicionar ferramentas de proteção

**31. Instale as dependências:**

```
npm install express-validator helmet express-rate-limit cors
npm install -D @types/cors
```

**32. Crie src/middlewares/validators.ts:**

- a) validarRegistro: nome (trim+notEmpty), email (isEmail+normalizeEmail), senha (isLength min 6)
- b) validarLogin: email (isEmail+normalizeEmail), senha (notEmpty)
- c) Adicionar .escape() no nome e texto de comentário (anti-XSS)
- d) Exportar validationResult

**33. Crie src/middlewares/rateLimiter.ts:**

- a) limiterGeral: 100 requisições / 15 minutos
- b) limiterLogin: 5 requisições / 15 minutos

**34. Atualize src/app.ts — adicionar na ORDEM correta:**

- a) helmet() (primeiro!)
- b) cors()
- c) limiterGeral
- d) Depois: parsers → session → rotas

**35. Aplique validators e limiters nas rotas:**

- a) POST /registro: validarRegistro + validationResult
- b) POST /login: validarLogin + limiterLogin + validationResult
- c) POST /comentar: validar texto com trim+notEmpty+escape

**36. Verificação final:**

**Vulnerabilidades corrigidas:**

- ☐ XSS: script aparece como texto
- ☐ Senhas no JSON são HASH (\$2b\$10\$...)
- ☐ Login: mensagem genérica + bcrypt.compare

- ☐ IDOR: não edita/remove de outro
- ☐ API /api/usuarios: sem senhas
- ☐ Admin: protegido por role
- ☐ httpOnly: true + secret forte

**Ferramentas instaladas:**

- ☐ F12 → Network → Headers: X-Content-Type-Options aparece (Helmet)
- ☐ POST com campos vazios → erros do express-validator
- ☐ 6x login errado → rate limit bloqueia
- ☐ document.cookie no Console → vazio

---

37. Middleware sanitizar() automático.

ts

```
export function sanitizar(req, res, next) {  
  for (const key in req.body) {  
    if (typeof req.body[key] === "string") {  
      req.body[key] = req.body[key].trim();  
    }  
  }  
  next();  
}
```

Para o escape() automático, usar uma lib como xss ou confiar no escape() do express-validator campo a campo.

---

38. CORS restrito para localhost:5173.

ts

```
app.use(cors({  
  origin: "http://localhost:5173",  
  credentials: true  
}));
```

---

39. Validação .custom() que checa email duplicado.

ts

```
body("email")  
  .isEmail()  
  .normalizeEmail()  
  .custom((email) => {  
    const users = carregarUsers();  
    const existe = users.find(u => u.email === email);  
    if (existe) throw new Error("Email já cadastrado");  
    return true;  
  })
```

## Checklist Final

### Vulnerabilidades:

- ☐ XSS: <%= %> + escape() + httpOnly
- ☐ Validação: todos os inputs no back-end
- ☐ bcrypt: hash no registro, compare no login
- ☐ IDOR: verificar userId em editar/remover
- ☐ API: nunca retornar senha/hash
- ☐ Mensagem genérica no login
- ☐ Stack trace: nunca enviar ao cliente

### Ferramentas:

- ☐ express-validator: validarRegistro + validarLogin
- ☐ validationResult checado em CADA rota
- ☐ Sanitização: trim() + normalizeEmail() + escape()
- ☐ helmet() registrado ANTES de tudo
- ☐ cors() configurado
- ☐ limiterGeral: 100/15min + limiterLogin: 5/15min
- ☐ Ordem: helmet → cors → rateLimit → parsers → session → rotas